CPS 379 · REINFORCEMENT LEARNING · FALL 2026 · FINAL PROJECT

Full Understanding of the Code

Budgeted Stochastic Scene Repair: fixing a tiny grid scene when edits can fail

Binayak Jha · solo project · code in `scene-repair/`

What this guide is for

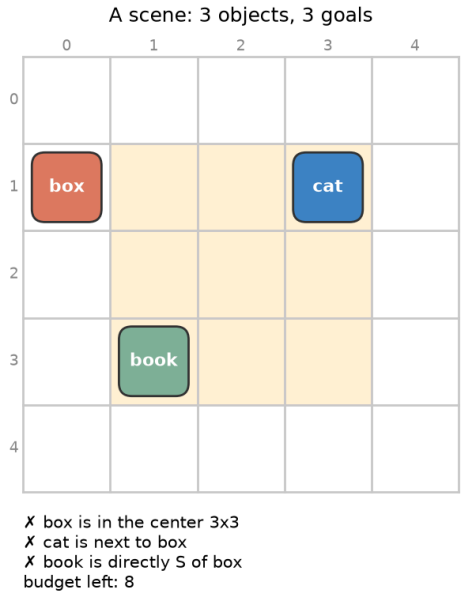
Read it once top to bottom and you can explain every file, every rule and every number in class. Each part starts with the idea in plain words, then shows the real code (copied automatically from the source files), then says what each important line does. Every number in this guide is read from the saved results files; none is typed by hand.

1. The problem in one paragraph
2. How the pieces fit
3. A walk through every file
4. The MDP mapped to code
5. How each policy works
6. How PPO works here, step by step
7. How to run it
8. The real results
9. Explain it in 30 seconds + professor questions
10. Limitations and deviations

1 The problem in one paragraph

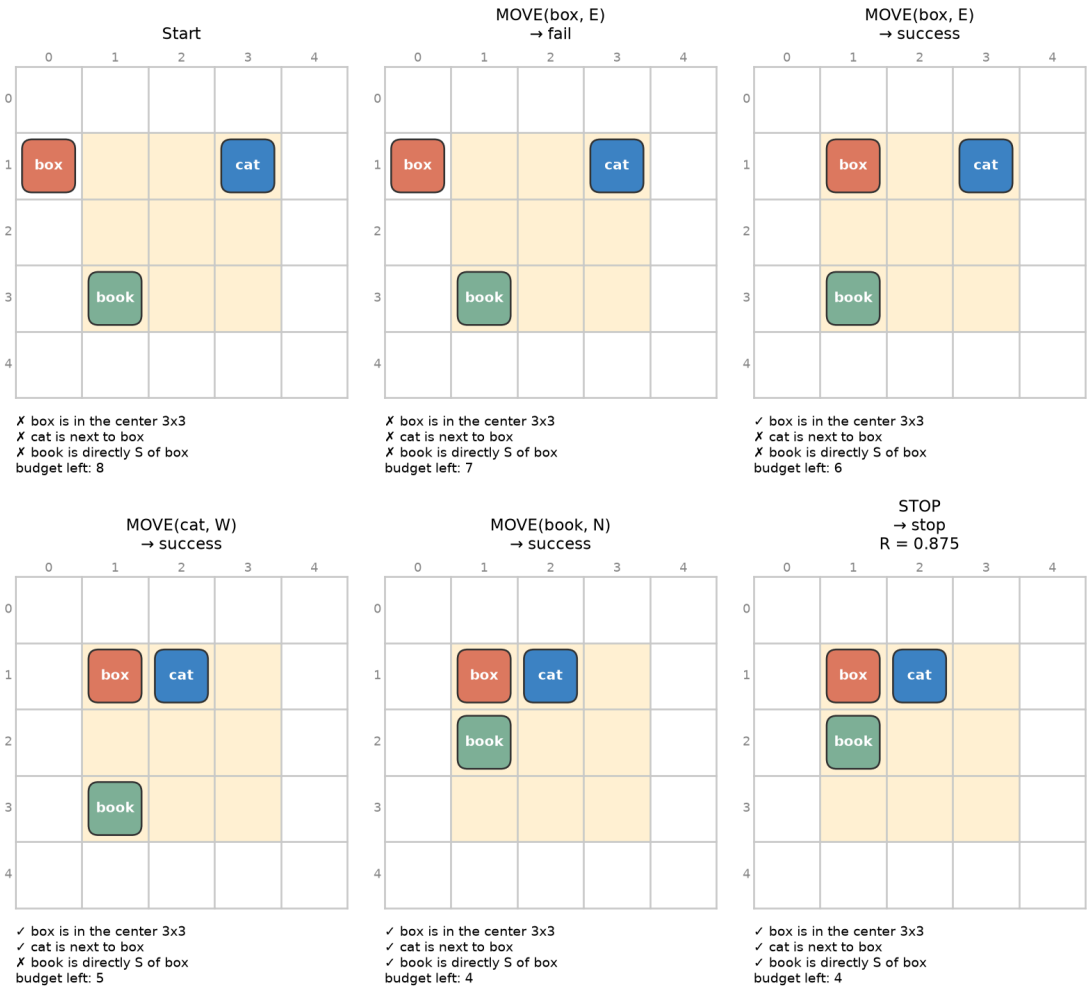
We have a **5×5 grid** with **3 to 5 labeled objects** (box, cat, book, lamp, cup), one per cell. We are given **2 to 4 goals** such as "box is in the center", "cat is next to box", or "book is directly south of box". The agent fixes the scene by moving objects one cell at a time (north, south, east, west). It can also **UNDO** its last move or **STOP**. The catch: **moves are unreliable**. A legal move works 70% of the time, lands in a random nearby cell 20% of the time ("jitter"), and does nothing 10% of the time ("fail"). The agent has a **budget of 8 actions**. At the end it gets one score: **R = s × (0.75 + 0.25 × budget left / 8)**, where **s** is the share of goals met. So: meet every goal, and do it with as few actions as possible.

> **Why this is an RL problem**
>
> Because edits can fail, you cannot just plan once and execute. After every action you must look at what really happened and decide again: retry, repair, undo, or stop. That is a sequential decision under uncertainty with a limited budget, which is exactly what RL is for.



The proposal's example scene. Shaded cells are the center 3×3. ✗ means the goal is not met yet.

The worked example from the proposal (it is also a unit test)

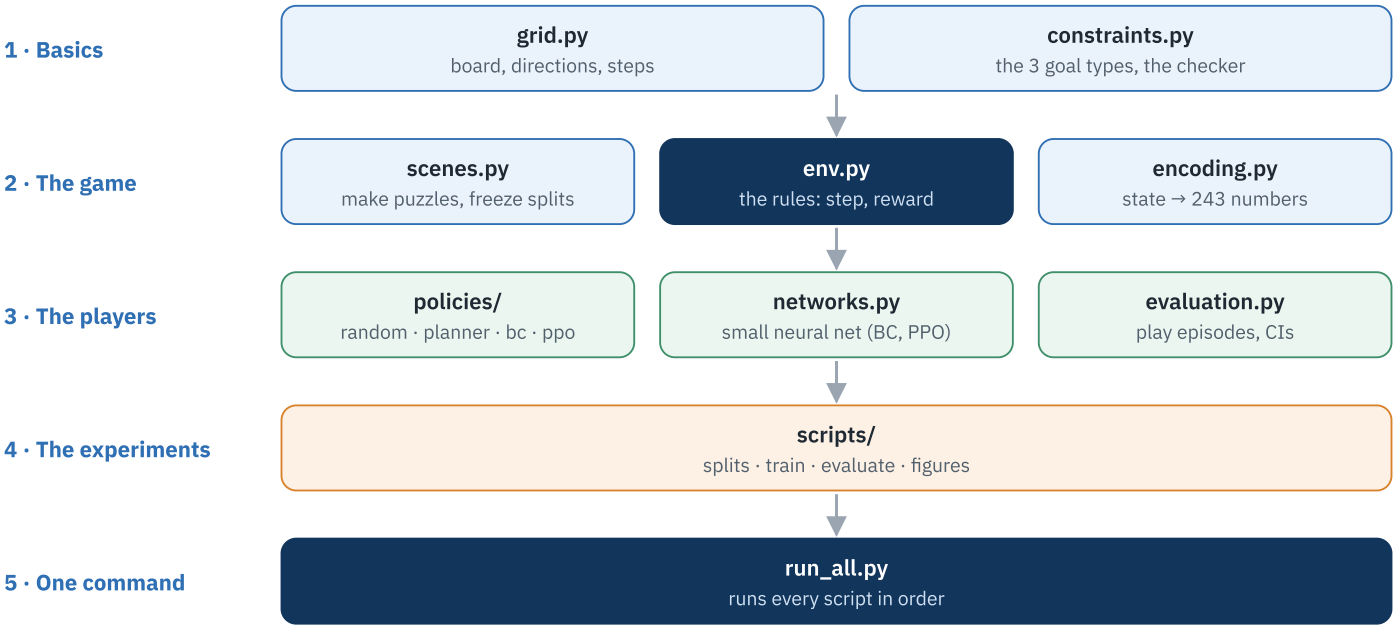


Box (0,1), cat (3,1), book (1,3). The first move fails, then three moves succeed, then STOP. Four actions were paid for, so R = 1 × (0.75 + 0.25 × 4/8) = 0.875. The test `test_worked_example_from_the_proposal_gives_0_875` forces exactly these outcomes and checks every number.

Coordinates: (x, y), x grows to the east (right), **y grows to the south (down)**, like rows on a page. So "north" means y − 1.

2 How the pieces fit

The project is a small Python package, `scene_repair/`, plus a few scripts that use it. Think of it as five layers. Each layer only uses the layers above it, so you can read the code top to bottom.



One episode, as a loop

```
obs, info = env.reset(seed=..., options={"scene": scene})   # new puzzle, budget 8, empty undo
buffer
done = False
while not done:
    action = policy.act(obs, info)          # every policy has this same one method
    obs, reward, done, truncated, info = env.step(action)
# reward is 0 on every step except the last one, where it is s × (0.75 + 0.25 × b/8)
```

The one design rule that keeps everything simple

Every policy answers the same question, `act(obs, info) → action number`. `obs` is the state as 243 numbers (for neural nets). `info["state"]` is the same state in readable form (for the planner). `info["action_mask"]` says which of the 22 actions exist. So the environment, the evaluator and the scripts never care which policy is playing.

3 A walk through every file

Every file starts with a docstring that says its one job. Here they are, with their sizes:

File	Lines	Its one job (first line of its docstring)
`run_all.py`	85	Reproduce EVERYTHING with one command.
`scene_repair/__init__.py`	12	Budgeted Stochastic Scene Repair: a small RL project for CPS 379.
`scene_repair/constraints.py`	71	The three kinds of goals ("constraints") and how to check them.
`scene_repair/drawing.py`	39	Draw a scene on a matplotlib axis: the 5x5 board, the center 3x3, the objects and goals.
`scene_repair/encoding.py`	138	Turn the readable state (C, L, b, m, h, o) into one flat list of numbers for a neural net.
`scene_repair/env.py`	184	The Gymnasium environment: the exact rules of Budgeted Stochastic Scene Repair.
`scene_repair/evaluation.py`	66	Playing episodes and summarising them: success rate, mean reward, actions used, and CIs.
`scene_repair/grid.py`	61	The board: a 5x5 grid, the four move directions and small position helpers.
`scene_repair/networks.py`	59	The small neural network shared by behavior cloning and PPO.
`scene_repair/policies/bc.py`	47	Behavior cloning (BC): copy the planner.
`scene_repair/policies/planner.py`	102	Adaptive replanner: the strong non-learning baseline.
`scene_repair/policies/ppo.py`	163	PPO (Proximal Policy Optimization), written out step by step.
`scene_repair/policies/random_policy.py`	13	Random policy: picks any allowed action uniformly at random. It is the "floor" to beat.
`scene_repair/scenes.py`	203	Making puzzles ("scenes") and splitting them into train / validation / test.
`scene_repair/settings.py`	65	All the experiment knobs in one place. run_all.py --quick swaps FULL for QUICK.
`scripts/evaluate.py`	129	Final evaluation on the frozen TEST set (run once, after all training and tuning).
`scripts/failure_analysis.py`	53	Why do episodes fail? For each policy on the main test set, sort failed episodes into:
`scripts/make_episode_gif.py`	74	Animate one real test episode played by PPO (seed 0) as a GIF plus a still strip.
`scripts/make_figures.py`	161	Make every figure in figures/ from the saved results (matplotlib only, no AI images).
`scripts/make_splits.py`	35	Freeze the validation, test and train-check scene sets as JSON files in data/.

`scripts/scene_stats.py`	36	Describe the frozen test scenes: objects, goals, and the shortest noise-free fix length.
`scripts/train_bc.py`	70	Train behavior cloning for one seed: collect planner demos, then fit the network.
`scripts/train_ppo.py`	63	Train PPO for one seed. Every few updates we check VALIDATION success and keep the best
`tests/helpers.py`	38	Test helpers: a fake random-number source so tests can force success, jitter or failure.
`tests/test_constraints.py`	35	Predicates use (x, y) with y growing SOUTH, exactly as the proposal says.
`tests/test_env.py`	171	The environment rules: collisions, noise, jitter, undo, budget, STOP and reward.
`tests/test_policies.py`	54	Policies: the planner solves the worked example; nets respect the action mask; PPO math.
`tests/test_scenes.py`	40	Reset generator: solvable, not already solved, within reach, and splits never overlap.

3.1 `grid.py`: the board

Positions are tuples `(x, y)`. A **layout** is a tuple of positions, one per object, so `layout[0]` is where the box is. Tuples cannot be changed, which means we can put layouts in sets (the planner needs that) and we never change a layout by accident.

scene_repair/grid.py

```
def step(cell, direction):
    dx, dy = DIRECTIONS[direction]
    return (cell[0] + dx, cell[1] + dy)
```

scene_repair/grid.py

```
def moore_neighbors(cell):
    x, y = cell
    result = []
    for dy in (-1, 0, 1):
        for dx in (-1, 0, 1):
            if dx == 0 and dy == 0:
                continue  # the cell itself is not its own neighbor
            neighbor = (x + dx, y + dy)
            if in_bounds(neighbor):
                result.append(neighbor)
    return result
```

- `DIRECTIONS["S"] = (0, 1)`: south adds 1 to y. This is the single place that fixes the "y grows south" rule.
- "Moore neighbors" just means the 8 touching cells, diagonals included. Jitter uses them.

3.2 `constraints.py`: the three goal types

scene_repair/constraints.py

```
def is_satisfied(constraint, layout):
    a = layout[constraint.i]
    if constraint.kind == "center":
        return 1 <= a[0] <= 3 and 1 <= a[1] <= 3
    b = layout[constraint.j]
    if constraint.kind == "adjacent":
        return manhattan(a, b) == 1
    if constraint.kind == "direction":
        return a == step(b, constraint.direction)
    raise ValueError(f"unknown constraint kind {constraint.kind}")
```

- **Center**: both x and y are 1, 2 or 3.
- **Adjacent**: Manhattan distance (|dx| + |dy|) is exactly 1, so diagonal does not count.
- **Direction**: object i must sit on the exact cell you get by stepping from j in direction d. `Direction(book, box, "S")` means book = box + (0, 1).
- `satisfaction_mask` runs this for every goal (that is **m**), and `fraction_satisfied` is **s**.

`moves_needed_at_least` gives, for one goal, the fewest moves that could possibly fix it. Each move shifts one object by one cell, so it can close a gap by at most 1. The planner uses this as its "estimate" (it never over-estimates, which is what A* needs).

3.3 `scenes.py`: making puzzles and keeping splits apart

scene_repair/scenes.py

```
def generate_scene(rng):
    while True:
        k_count = int(rng.integers(3, MAX_OBJECTS + 1))
        cells = all_cells()
        picks = rng.choice(len(cells), size=k_count, replace=False)
        witness = tuple(cells[p] for p in picks)
        how_many = int(rng.integers(2, MAX_CONSTRAINTS + 1))
        constraints = _pick_constraints(rng, witness, how_many)
        if constraints is None:
            continue
        layout = witness
        for _ in range(int(rng.integers(1, MAX_SCRAMBLE_MOVES + 1))):
            k, d = legal_moves(layout)[rng.integers(len(legal_moves(layout)))]
            layout = move_object(layout, k, step(layout[k], d))
        if all(satisfaction_mask(constraints, layout)):
            continue  # already solved: reject, as the proposal says
        return Scene(constraints, layout, witness)
```

- **Witness first.** Drop K objects on random cells, then pick goals that are *true* there. So every puzzle has a known solution.
- **Scramble.** Make 1 to 6 random legal moves with no noise. Playing them backwards solves the puzzle, so a noise-free fix always fits in the budget.
- **Reject solved scenes**, as the proposal says (`continue` means "try again").

- `_pick_constraints` always includes at least one goal between two objects and never puts two goals on the same pair (so "cat south of box" and "cat next to box" never both appear; one would be redundant).

scene_repair/scenes.py

```
def split_of_template(key):
    bucket = int(hashlib.md5(key.encode()).hexdigest(), 16) % 10
    if bucket <= 6:
        return "train"
    if bucket == 7:
        return "val"
    return "test"
```

- A **template** is the exact goal list, e.g. `center(0)|adjacent(0,1)|direction(2,0,S)`.
- We hash it into 10 buckets: 0–6 train, 7 validation, 8–9 test. So the **same goal list never appears in two splits**. On top of that, each split draws from its own seed range (train 0+, val 1,000,000,000+, test 2,000,000,000+).
- Validation (300 scenes, 243 templates) and test (500 scenes, 420 templates) are frozen as JSON in `results/data/`. Training scenes are drawn fresh forever.

3.4 `env.py`: the rules of the game (the most important file)

It is a normal Gymnasium environment: `reset()` starts an episode, `step(action)` returns the five values `obs, reward, terminated, truncated, info`.

scene_repair/env.py

```
def step(self, action):
    assert not self.done, "episode is over; call reset()"
    action = int(action)

    if action == STOP:
        self.outcome = "stop"
        self.done = True  # STOP is free: budget is not touched
    else:
        self.budget -= 1  # every MOVE and UNDO costs one unit, whatever happens
        if action == UNDO:
            self._do_undo()
        else:
            self._do_move(action)
        if self.budget == 0:
            self.done = True  # out of budget: the episode is forced to end

    reward = 0.0
    if self.done:
        s = fraction_satisfied(self.constraints, self.layout)
        reward = terminal_reward(s, self.budget, self.start_budget)
    # terminated=True means the episode really ended; we never use truncation
    return self._obs(), reward, self.done, False, self._info()
```

- **STOP** is free: it does not touch the budget. It ends the episode.
- **MOVE and UNDO always cost 1**, and the charge happens *before* we look at the outcome. So a failed or illegal move still costs.
- If the budget reaches 0, the episode is forced to end.

- **Reward only at the end**: `reward` stays 0.0 unless `self.done`. That is the proposal's "no intermediate reward".
- `truncated` is always False: the budget rule is part of the task, so running out is a real ending (terminated), not a time-limit cut.

scene_repair/env.py

```
def _do_move(self, action):
    k, d = divmod(action, 4)
    before = self.layout
    self.undo_buffer = before  # every MOVE saves the pre-move layout (length-one buffer)

    if k >= len(self.layout):
        self.outcome = "invalid"  # padded slot: this object does not exist
        return
    u = before[k]                          # where the object is now
    v = step(u, DIRECTION_NAMES[d])        # where we want it to go
    if not in_bounds(v) or v in before:
        self.outcome = "invalid"           # off the board or onto another object
        return

    p_success, p_jitter, _ = self.noise
    roll = self.np_random.random()
    if roll < p_success:
        self.layout = move_object(before, k, v)
        self.outcome = "success"
    elif roll < p_success + p_jitter:
        self.outcome = "jitter"
        candidates = jitter_cells(before, u, v)
        if candidates:  # pick one at random; if none, nothing moves
            pick = candidates[self.np_random.integers(len(candidates))]
            self.layout = move_object(before, k, pick)
    else:
        self.outcome = "fail"
```

- First, **every MOVE saves the layout from before it** in the undo buffer `h` (length one: the new save replaces the old).
- `divmod(action, 4)` turns an action number into (object k, direction d). Example: action 9 = 4×2 + 1 = MOVE(book, S).
- **Illegal** = target off the board or already occupied. Nothing moves; outcome "invalid".
- One random number decides the outcome: below 0.7 → success, 0.7 to 0.9 → jitter, above 0.9 → fail.

scene_repair/env.py

```
def jitter_cells(layout_before, u, v):
    occupied = set(layout_before)
    return [c for c in moore_neighbors(v) if c not in occupied and c != u]
```

- Jitter lands on an **empty**, on-board, 8-way neighbor of the *intended* cell v, judged on the grid before the move. The starting cell u is excluded.
- If that list is empty, nothing moves (tested by `test_jitter_with_no_candidates_leaves_layout_unchanged`).

scene_repair/env.py

```
def _do_undo(self):
    if not self.allow_undo or self.undo_buffer is None:
        self.outcome = "undo_empty"  # charged no-op
        return
    self.layout = self.undo_buffer  # deterministic restore
    self.undo_buffer = None         # buffer is cleared after use
    self.outcome = "undo"
```

- UNDO restores the saved layout **with no noise** and empties the buffer. With an empty buffer it does nothing (still charged). There is never a refund.

> **Design choice: padding to 22 actions**
>
> Scenes have 3 to 5 objects, but a neural net needs a fixed number of outputs. So there are always 4 × 5 + 2 = **22 actions**: 0–19 are MOVE(k, d) = 4k + d, 20 is UNDO, 21 is STOP. Actions for objects that do not exist are switched off in `info["action_mask"]`; if one is picked anyway it counts as an invalid, charged move. Off-board and occupied-target moves are **not** masked: they are real (bad) choices in the MDP, and the agent must learn to avoid them.

3.5 `encoding.py`: turning the state into numbers

A neural net needs a fixed-length list of numbers. The state (C, L, b, m, h, o) becomes **243 numbers**:

Block	What it holds	Size
Objects	for each of 5 slots: present?, x as one-hot, y as one-hot, which of N/S/E/W is legal now, "pull" (dx, dy)	5 × 17 = 85
Goals C, mask m	for each of 4 slots: present?, kind, object i, object j, direction, met?, gap (dx, dy)	4 × 21 = 84
Budget b	one-hot of 0..8, and b / 8	10
Undo buffer h	has a saved layout?, then the saved layout (5 × 11)	56
Last outcome o	one-hot over start / success / jitter / fail / invalid / undo / undo_empty / stop	8

One-hot = a row of zeros with a single 1 marking the value. x = 3 becomes 0 0 0 1 0.

scene_repair/encoding.py

```
def object_pulls(state):
    pulls = np.zeros((MAX_OBJECTS, 2), dtype=np.float32)
    for c, ok in zip(state["constraints"], state["mask"]):
        if ok:
            continue
        gap = goal_gap(c, state["layout"])
        pulls[c.i] += gap
        if c.j >= 0:
            pulls[c.j] -= gap
    return pulls
```

- **Gap** (per goal): how far object i still has to travel, in x and in y, to meet the goal.

- **Pull** (per object): the gaps of all its unmet goals added up, so it says "which way this object wants to go". If the object is the second one (j) in a goal, the gap counts negative, since moving j the other way closes the same gap.
- These are **hints computed only from C and L**. They add no information the agent did not already have; they only save the small net from learning arithmetic. Section 6.4 shows why we added them: without the pull hint PPO learned very slowly.

3.6 `networks.py` : the small neural net

scene_repair/networks.py

```
class PolicyValueNet(nn.Module):
    def __init__(self, obs_size, num_actions, hidden=128):
        super().__init__()
        self.body = nn.Sequential(
            nn.Linear(obs_size, hidden), nn.Tanh(),
            nn.Linear(hidden, hidden), nn.Tanh(),
        )
        self.policy_head = nn.Linear(hidden, num_actions)
        self.value_head = nn.Linear(hidden, 1)

    def forward(self, obs, action_mask):
        features = self.body(obs)
        logits = self.policy_head(features)
        logits = logits.masked_fill(~action_mask, -1e9)  # forbidden actions get probability 0
        value = self.value_head(features).squeeze(-1)
        return logits, value

    def distribution(self, obs, action_mask):
        logits, value = self.forward(obs, action_mask)
        return torch.distributions.Categorical(logits=logits), value
```

- The **body** is 2 layers of 128 units with tanh. Both heads read from it.
- The **policy head** gives 22 scores ("logits"); softmax turns them into probabilities.
- `masked_fill(~action_mask, -1e9)` : switched-off actions get a score of −1,000,000,000, so their probability is 0.
- The **value head** guesses the final reward from this state. Only PPO uses it. BC uses the same class and ignores it.
- `NetPolicy` wraps a trained net so it has the usual `act(obs, info)` . In evaluation it is **greedy**: it takes the most likely action.

3.7 `evaluation.py`: playing and scoring

scene_repair/evaluation.py

```
def bootstrap_ci(records, key, num_resamples=2000, seed=0):
    by_scene = {}
    for r in records:
        by_scene.setdefault(r["scene_index"], []).append(float(r[key]))
    per_scene = np.array([np.mean(v) for v in by_scene.values()])
    rng = np.random.default_rng(seed)
    picks = rng.integers(0, len(per_scene), size=(num_resamples, len(per_scene)))
    means = per_scene[picks].mean(axis=1)
    return float(per_scene.mean()), float(np.percentile(means, 2.5)), float(np.percentile(means,
97.5))
```

- A **95% CI** (confidence interval) is a range that likely holds the true average.
- **Bootstrap**: resample the test scenes with replacement 2,000 times, compute the average each time, and keep the middle 95%.
- We resample **whole scenes**, not single episodes, because the 4 rollouts of one scene are related. For BC and PPO, each (seed, scene) pair is one unit, so seed-to-seed variation is inside the interval too.

3.8 The scripts and the tests

- `scripts/make_splits.py` freezes validation, test and a "train-check" set (training scenes used only to measure overfitting).
- `scripts/train_bc.py`, `scripts/train_ppo.py` train one seed each and keep the network that does best on **validation**.
- `scripts/evaluate.py` plays every policy on the same test scenes with the same rollout seeds and writes `results/test_results.json` and `results/results_table.md`.
- `scripts/make_figures.py` and `scripts/make_episode_gif.py` draw everything in `figures/` with matplotlib.
- `tests/`: **29 passed in 2.23s**. They cover predicates and coordinates, collisions, failure, jitter candidates and the empty fallback, undo (restore, clear, length one, empty), budget charging, STOP, forced termination, terminal reward, padding and masking, the no-UNDO switch, the noise rates, the reset generator, split separation, the planner, and the PPO advantage math. `tests/helpers.py` has a `FakeRandom` that lets a test force "success", "jitter" or "fail".

4 The MDP mapped to code

An **MDP** (Markov decision process) is the formal name for "states, actions, how actions change the state, and rewards". Here is each piece and the exact line of code that implements it.

MDP piece	In the proposal	In the code
State	(C, L, b, m, h, o), all observed	`env.state()` returns all six; `encoding.encode` turns them into 243 numbers
Goals C	Center3x3, ManhattanAdj, CardinalAdj	`Constraint` + `is_satisfied` in `constraints.py`
Layout L	one cell per object, (x, y), y south	`self.layout`, a tuple of (x, y); `grid.DIRECTIONS`
Budget b	B_0 = 8; MOVE/UNDO cost 1; STOP free	`self.budget -= 1` in `step`, skipped for STOP
Mask m	recomputed from C and L	`satisfaction_mask(constraints, layout)`
Undo buffer h	length one, starts empty	`self.undo_buffer`; set in `_do_move`, used and cleared in `_do_undo`
Outcome o	success / jitter / failure / invalid / undo	`self.outcome`, one of 8 names in `env.OUTCOMES`
Actions	MOVE(k, d), UNDO, STOP: 4K + 2	22 numbers: 4k + d, 20 = UNDO, 21 = STOP, plus `action_mask`
Transition	0.7 / 0.2 / 0.1; illegal = no change	`_do_move` and `jitter_cells`
Reward	R = s(0.75 + 0.25 b/8) at the end only	`terminal_reward`, called only when `self.done`
End of episode	STOP, or b = 0	`self.done = True` in both places; `terminated`
Initial states	scrambled witness, ≤ 6 moves, not solved	`scenes.generate_scene`

> **Why the state is "Markov" (no hidden memory needed)**
>
> Everything that affects the future is in (C, L, b, m, h, o). The undo buffer is part of the state, so the agent never needs to remember its history. That is why a simple feed-forward net (no memory) is enough for both BC and PPO.

Episode length is at most 9 steps (8 paid actions, then the forced end; or STOP earlier). The test scenes need on average **1.85** noise-free moves to fix (shortest fix lengths: 1 move: 231, 2 moves: 157, 3 moves: 78, 4 moves: 26, 5 moves: 7, 6 moves: 1). Scenes: 3 objects: 78, 4 objects: 139, 5 objects: 283; 2 goals: 223, 3 goals: 174, 4 goals: 103.

5 How each policy works, in plain words

Random

Picks any allowed action uniformly. It is the floor: anything useful must beat it.

scene_repair/policies/random_policy.py

```
def act(self, obs, info):
    allowed = np.flatnonzero(info["action_mask"])  # real objects, UNDO, STOP
    return int(self.rng.choice(allowed))
```

Adaptive replanner (strong non-learning baseline)

In one line: pretend moves never fail, find the shortest fix, do its *first* step, look at what really happened, and plan again.

- **A* search** explores layouts in order of "moves so far + an estimate of moves left". The estimate is the largest `moves_needed_at_least` over the goals; it never over-estimates, so the first full fix A* finds is a shortest one.
- It may also use **UNDO** as a first step, since UNDO is the one noise-free action.
- It never searches deeper than the budget left, and stops after 20,000 expanded layouts. If no full fix fits, it heads for the best partial layout it saw. When the scene is solved it says STOP.
- It uses the same visible goals and the same exact checker as everyone else, but it reads the readable state instead of the 243 numbers.

scene_repair/policies/planner.py

```
def plan(state, max_expansions=MAX_EXPANSIONS, allow_undo=True):
    constraints, start, budget = state["constraints"], state["layout"], state["budget"]
    b0 = state["start_budget"]

    def score(layout, used):  # the noise-free reward if we stopped at this layout
        return terminal_reward(fraction_satisfied(constraints, layout), budget - used, b0)

    came_from = {start: None}   # layout -> (previous layout, action)
    cost = {start: 0}
    best_layout, best_score = start, score(start, 0)
    tie = itertools.count()     # breaks ties in the heap without comparing layouts
    heap = [(lower_bound(constraints, start), 0, next(tie), start)]
    expansions = 0

    while heap and expansions < max_expansions:
        _, g, _, layout = heapq.heappop(heap)
        if g > cost[layout]:
            continue  # stale heap entry; a shorter way here was already found
        if all(satisfaction_mask(constraints, layout)):
            best_layout = layout  # A* pops goals in order of cost, so this one is shortest
            break
        expansions += 1
        if g == budget:
            continue  # no budget left to go deeper
        options = list(successors(layout))
        if g == 0 and allow_undo and state["undo_buffer"] is not None:
            options.append((UNDO, state["undo_buffer"]))
        for action, nxt in options:
            if nxt not in cost or g + 1 < cost[nxt]:
                cost[nxt] = g + 1
                came_from[nxt] = (layout, action)
                s = score(nxt, g + 1)
                if s > best_score:
                    best_layout, best_score = nxt, s
                heapq.heappush(heap, (g + 1 + lower_bound(constraints, nxt), g + 1, next(tie), nxt))

    # walk back from the chosen layout to the start to read off the actions
    actions = []
    layout = best_layout
    while came_from[layout] is not None:
        layout, action = came_from[layout]
        actions.append(action)
    return actions[::-1], expansions
```

- `heap` is a priority queue: `heappop` always gives the layout with the smallest f = g + estimate.
- `came_from` remembers how we reached each layout, so at the end we walk backwards to read off the action list.
- `if g == budget: continue` stops the search from planning more moves than we can pay for.

Behavior cloning (BC)

Copy the planner. We let the planner play 6,000 training episodes *with noise* per seed and save every (state, planner's action) pair: 21,493, 21,834, 21,463 pairs for seeds 0, 1, 2. Because moves fail and jitter, these demos include messy recovery states, not just perfect paths. Then we train the network as a classifier ("given

this state, which action did the planner pick?") with cross-entropy loss. It is **reactive**: it only looks at the current state and does no search.

scene_repair/policies/bc.py

```
def train_bc_epoch(net, optimizer, observations, masks, actions, batch_size, rng):
    order = rng.permutation(len(actions))
    losses = []
    net.train()
    for start in range(0, len(order), batch_size):
        idx = order[start:start + batch_size]
        logits, _ = net(torch.as_tensor(observations[idx]), torch.as_tensor(masks[idx]))
        loss = torch.nn.functional.cross_entropy(logits, torch.as_tensor(actions[idx]))
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        losses.append(loss.item())
    net.eval()
    return float(np.mean(losses))
```

PPO

Learns from its **own** experience and the final reward only. No demos, no search. Next section explains it step by step.

6 How PPO works here, step by step

> **The idea in four sentences**
>
> Play a batch of episodes with the current policy. For each action, ask "did things turn out better or worse than my value head expected?" That difference is the **advantage**. Make good-advantage actions more likely and bad ones less likely, but only by a little each time (that is the "proximal" part). Repeat.

6.1 Play

32 environments run side by side for 64 steps each, so one batch is 2,048 steps. Actions are **sampled** from the policy's probabilities (so it explores). For each step we save the observation, mask, action, the action's log-probability, the value guess, the reward and whether the episode ended. A finished environment restarts at once on a new random training scene.

6.2 Score: advantages with GAE

scene_repair/policies/ppo.py

```
def compute_advantages(rewards, values, dones, last_values, gamma, lam):
    advantages = np.zeros_like(rewards)
    running = np.zeros(rewards.shape[1], dtype=np.float32)
    for t in reversed(range(len(rewards))):
        next_values = last_values if t == len(rewards) - 1 else values[t + 1]
        not_done = 1.0 - dones[t]  # after a finished episode, the future is worth 0
        delta = rewards[t] + gamma * next_values * not_done - values[t]
        running = delta + gamma * lam * not_done * running
        advantages[t] = running
    returns = advantages + values
    return advantages, returns
```

- `delta` = reward + value of next state − value of this state: a one-step "surprise".
- GAE adds up these surprises going backwards, shrinking older ones by λ = 0.95. With γ = 1 and λ = 1 this would be exactly "final reward − value guess" (a unit test checks that).
- `not_done` stops the sum at the end of an episode, so one puzzle's result never leaks into the next.
- γ (gamma, the discount) = 1 because episodes are at most 9 steps and only the final reward matters; the budget term already rewards being quick.

6.3 Learn: the clipped objective

scene_repair/policies/ppo.py

```
def ppo_update(net, optimizer, batch, cfg):
    obs, masks, actions, old_logp, advantages, returns = batch
    # normalise advantages: same signal, steadier step sizes
    advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)
    stats = []
    for _ in range(cfg.epochs):
        order = torch.randperm(len(actions))
        for start in range(0, len(order), cfg.minibatch_size):
            idx = order[start:start + cfg.minibatch_size]
            dist, value = net.distribution(obs[idx], masks[idx])
            logp = dist.log_prob(actions[idx])
            ratio = torch.exp(logp - old_logp[idx])
            unclipped = ratio * advantages[idx]
            clipped = torch.clamp(ratio, 1 - cfg.clip, 1 + cfg.clip) * advantages[idx]
            policy_loss = -torch.min(unclipped, clipped).mean()
            value_loss = ((value - returns[idx]) ** 2).mean()
            entropy = dist.entropy().mean()
            loss = policy_loss + cfg.value_coef * value_loss - cfg.entropy_coef * entropy
            optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(net.parameters(), cfg.max_grad_norm)
            optimizer.step()
            stats.append([policy_loss.item(), value_loss.item(), entropy.item()])
    policy_loss, value_loss, entropy = np.mean(stats, axis=0)
    return {"policy_loss": policy_loss, "value_loss": value_loss, "entropy": entropy}
```

- `ratio` = new probability of the action ÷ old probability. 1 means "unchanged".
- **Clipping**: `clamp(ratio, 0.8, 1.2)` and the `min` mean the update gets no extra credit for moving the probability more than 20%. That keeps each update small and stable.
- **Value loss** trains the value head to predict the real return.
- **Entropy bonus** (coefficient 0.03) rewards keeping some randomness, so the policy keeps exploring and does not lock in early.
- Advantages are normalised (mean 0, std 1) for steadier step sizes. Gradients are clipped at norm 0.5. Learning rate starts at 0.001 and shrinks linearly to 0.
- Each batch is used for 4 passes ("epochs") in mini-batches of 512.

6.4 How PPO was tuned (validation only)

All tuning used seed 0 and the frozen **validation** set; the test set was never looked at. Full notes and logs are in `results/tuning/`.

- **Round 1** (no pull hint): every setting was slow, the best reached 0.385 validation success after about 0.7M steps. We saw that the net had to learn "goal is east of object k → pick MOVE(k, E)" from raw one-hot positions.
- **Fix**: add the per-object pull hint (computed from C and L only).
- **Rounds 2–3** (with the hint): settings reached 0.865–0.915 best validation success. All were within the noise of a 200-scene check, so we picked the steadiest: learning rate 0.001 annealed to 0, entropy 0.03, hidden size 128.

- **Final runs**: 3,000,320 environment steps per seed, 3 seeds (9,000,960 steps total), about 11.8 minutes per seed on one CPU core. We checked validation every 25 updates and kept the best network.

> **Why not just train on the test set?**
>
> Picking settings or checkpoints by test score would make the test score optimistic. So: tune on validation, choose checkpoints on validation, and look at test once at the end.

7 How to run it

```
cd scene-repair
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python -m pytest -q tests      # the unit tests (a few seconds)
python run_all.py --quick      # tiny end-to-end check, about 1 minute, writes results/quick/
python run_all.py              # the full study, about 18 minutes on 8 CPU cores
python docs/build_guide.py     # rebuild this guide from the saved results
```

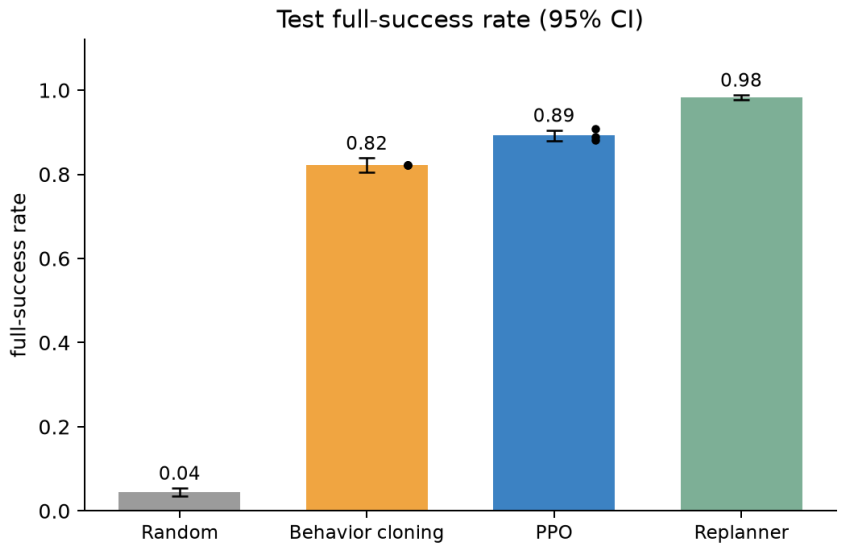
Single pieces: `python scripts/train_ppo.py --seed 1`, `python scripts/train_bc.py --seed 0 --variant no_undo`, `python scripts/evaluate.py --variants main`. All knobs live in `scene_repair/settings.py`.

Hardware used for the numbers below: Intel(R) Xeon(R) Processor, 8 cores, no GPU, Python 3.13.5, PyTorch 2.15.0+cpu. The full run took 18 minutes in total, of which 17 minutes were training (up to 7 jobs at once).

8 The real results

Test set: **500 scenes × 4 rollouts = 2,000 episodes** per policy (per seed for BC and PPO), same scenes and rollout seeds for every policy. Brackets are 95% bootstrap CIs. "Actions used" counts paid actions (STOP is free).

Policy	Full success	Mean reward R	Goals met (s)	Actions used	UNDOs / ep.
Random	4.4% [3.5, 5.4]	0.280 [0.267, 0.292]	0.346 [0.330, 0.361]	6.37 [6.25, 6.48]	0.34
Behavior cloning	82.3% [80.6, 83.9]	0.829 [0.820, 0.838]	0.916 [0.907, 0.925]	3.42 [3.31, 3.53]	0.01
PPO	89.3% [88.0, 90.5]	0.870 [0.863, 0.877]	0.953 [0.946, 0.959]	2.98 [2.89, 3.06]	0.00
Adaptive replanner	98.3% [97.7, 98.9]	0.912 [0.907, 0.917]	0.993 [0.990, 0.995]	2.64 [2.52, 2.76]	0.01



Test full-success rate. Error bars are 95% CIs; black dots are the 3 training seeds.

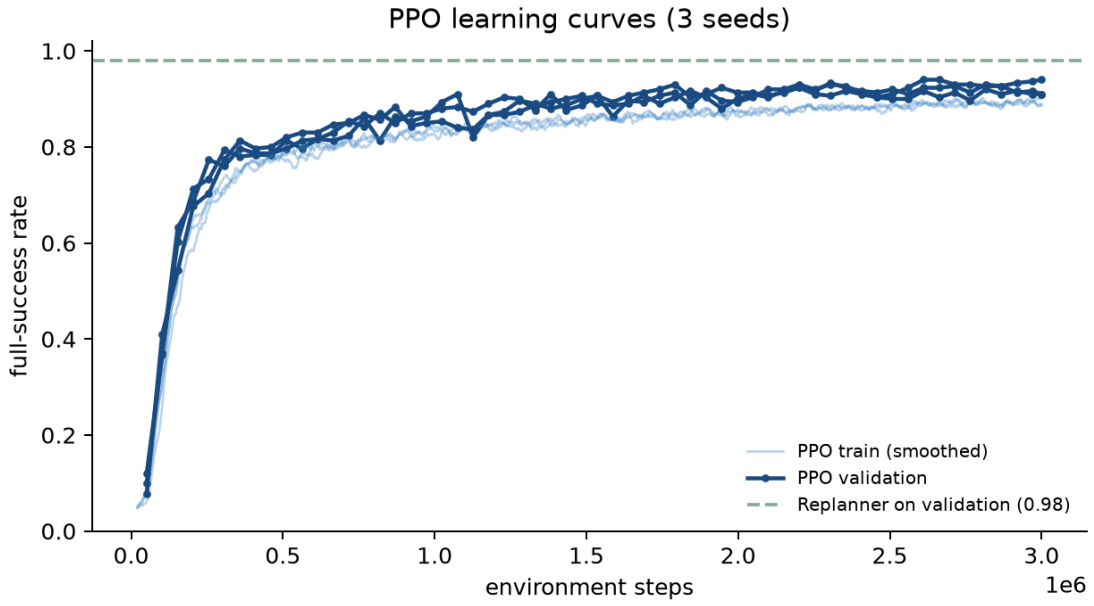
What this means

- **The replanner wins** (98.3% full success). The proposal said a planner win would be a valid result, and it is the honest result here. It has an exact model of the rules (minus the noise) and searches, and these puzzles are short (a 1.85-move fix on average), so search is cheap and strong.
- **PPO is second** (89.3%), clearly ahead of behavior cloning (82.3%); their CIs do not overlap. PPO learned only from the final reward, no demos, no search.
- **BC is below its own teacher.** It copies the planner's actions but cannot search, so small copying mistakes add up, and once it makes one it reaches states the planner rarely visited.
- **Random** almost never solves a scene (4.4%), which shows the task is not trivial.
- **Efficiency:** the planner uses 2.64 paid actions on average, PPO 2.98, BC 3.42. Fewer actions means more budget left, which is why mean R follows the same order.
- **UNDO is almost never used** by any trained or planning policy (last column). After a jitter, a normal move usually repairs things just as cheaply as UNDO.

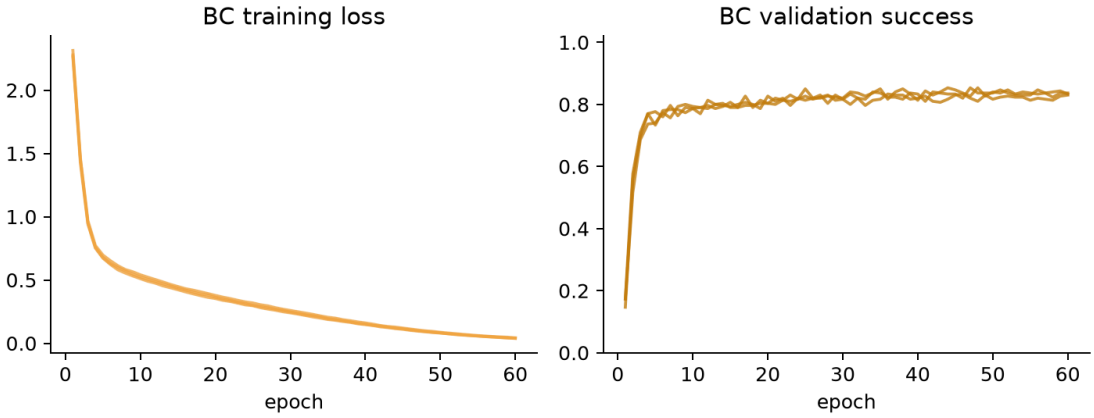
Each training seed, and train vs validation (overfitting check)

Policy	Seed	Test success	Test mean R	Train-check success	Val success
Behavior cloning	0	82.3%	0.828	92.7%	85.0%
Behavior cloning	1	82.2%	0.830	86.3%	85.3%
Behavior cloning	2	82.2%	0.830	84.0%	83.3%
PPO	0	88.9%	0.872	90.0%	91.7%
PPO	1	90.8%	0.876	91.0%	92.7%
PPO	2	88.2%	0.864	89.7%	91.7%

All three seeds land close together for both learned policies, so the result is not one lucky run. **Overfitting check:** BC does noticeably better on training scenes than on validation for seed 0 (it memorises its demos; its training loss keeps falling in the curve below while validation success stops improving). PPO scores about the same on training, validation and test scenes, so it generalises to goal lists it never saw in training. (Validation here uses different rollout seeds from the ones used to pick checkpoints, which is why these numbers differ slightly from the training logs.)



PPO success while training (light, smoothed) and on validation (dark dots), 3 seeds. Dashed: the replanner on the same validation scenes.

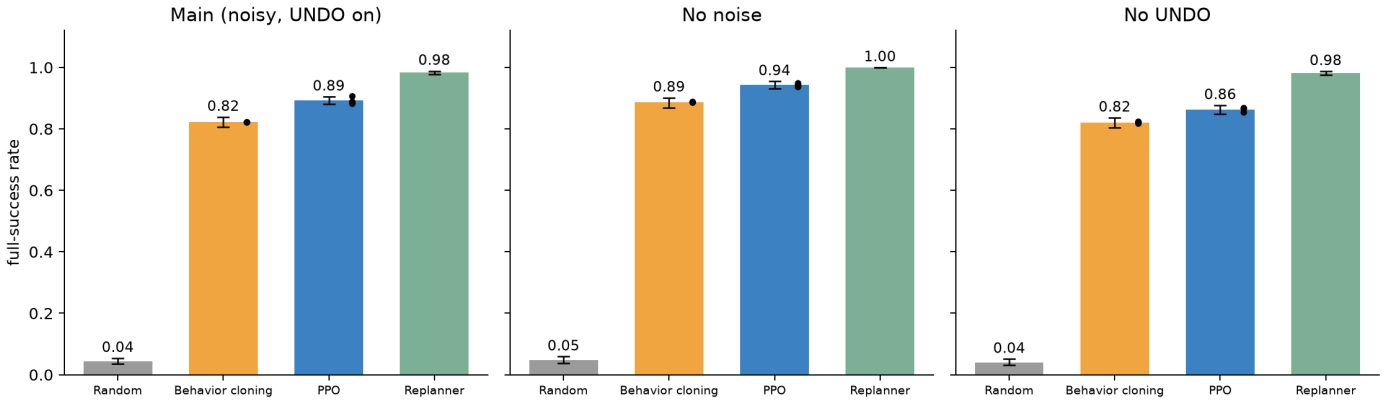


BC: training loss keeps falling while validation success flattens out early.

Ablations: where does the difficulty come from?

Each setting retrains BC and PPO from scratch (3 seeds; PPO for 2,000,000 steps) and re-runs the planner and random on the same test scenes.

Setting	Random	Behavior cloning	PPO	Adaptive replanner
Main (noise on, UNDO on)	4.4% [3.5, 5.4]	82.3% [80.6, 83.9]	89.3% [88.0, 90.5]	98.3% [97.7, 98.9]
No noise	4.9% [3.9, 5.9]	88.7% [86.9, 90.1]	94.4% [93.2, 95.5]	100.0% [100.0, 100.0]
No UNDO	4.0% [3.1, 5.1]	82.1% [80.3, 83.6]	86.3% [84.9, 87.7]	98.2% [97.5, 98.8]



Full success on the test set for each setting.

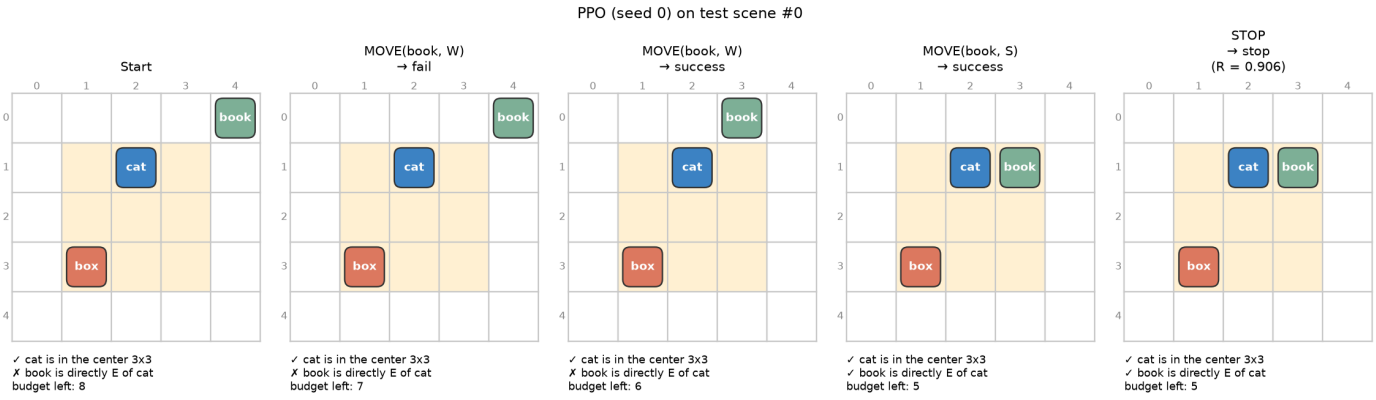
What the ablations say

- **Turning noise off** helps everyone: planner 98.3% → 100.0%, PPO 89.3% → 94.4%, BC 82.3% → 88.7%. So noise costs the planner about 2 points and the learners about 5–6 points. But even with no noise, the learned policies still miss some scenes, so part of their gap is in *learning* the mapping from state to action, not only in handling noise.
- **Removing UNDO** changes nothing for the planner (98.2%) or BC (82.1%). PPO is lower (86.3%), but main-setting PPO used UNDO essentially never, so this drop is **not** caused by losing UNDO. The likely reason is that ablation PPO runs trained for 2,000,000 steps instead of 3,000,000 (to save compute) and the run-to-run spread. Honest reading: **UNDO does not matter in this task** as designed.

Compute, reported separately (as the proposal promised)

- **Planner search:** 0.30 ms per decision on average; 60,000 layouts expanded over 7,239 test decisions (limit 20,000 per decision).
- **BC demonstrations:** 6,000 planner episodes per seed (21,493, 21,834, 21,463 state-action pairs); about 0.5 minutes per seed including demo collection.
- **PPO:** 3,000,320 environment steps per seed, no demos, no search; about 11.8 minutes per seed.

One real test episode played by PPO



The first test scene where a move went wrong and PPO still finished. Animated version: `figures/ppo_episode.gif`.

9 Explain it in 30 seconds

Say this

"I built a small RL benchmark: fix a 5×5 grid scene so it meets goals like 'book directly south of box', with only 8 paid actions, where each move works 70% of the time, jitters 20% and fails 10%. The agent sees everything and gets one reward at the end: the share of goals met, scaled up for leftover budget. I compared four players on the same 500 held-out scenes: random, an A* replanner that replans after every outcome, behavior cloning of that planner, and PPO. The planner wins with 98.3% full success; PPO, learning only from the end reward, gets 89.3% and beats behavior cloning at 82.3%. Turning noise off helps everyone, and turning UNDO off changes almost nothing, so here the hard part is noise plus learning the state-to-action mapping, not undoing."

Likely professor questions, with answers

Q: Why does the planner beat PPO? Isn't that bad for an RL project?
A: No. The proposal said a planner win is a valid result. The planner is given a near-perfect model of the rules and the puzzles are short, so search is cheap and very strong. PPO had to discover everything from a single end-of-episode number. Getting to 89.3% that way, and beating imitation of the planner, is the interesting part.

Q: Why does PPO beat behavior cloning, when BC copies the better player?
A: BC only copies; it does no search. Small mistakes put it in states the planner rarely reached, and it has no signal for what to do there. PPO trains on its own mistakes and is graded on the real reward, so it learns to recover.

Q: Why PPO and not DQN?
A: The action set is small and discrete, which suits both. PPO handles the action mask cleanly (masked softmax), works well with on-policy terminal rewards, and has fewer moving parts to explain (no replay buffer, no target network). The proposal committed to PPO.

Q: Is the 'pull' hint cheating?
A: No. It is computed only from the goals C and the layout L, which the agent already sees, and BC gets the exact same numbers. It is feature engineering, like normalising inputs. Without it PPO reached only 0.385 validation success in tuning; that is documented in results/tuning/.

Q: How do you know you did not tune on the test set?
A: Every setting and every checkpoint was chosen with the validation set. The test set was used once, in scripts/evaluate.py, after training finished. Splits are disjoint by seed range and by goal-list template.

Q: Why is the reward only at the end? Isn't that hard for RL?
A: That is what the proposal specified: no shaping. Episodes are at most 9 steps, so credit assignment is short. The value head and GAE turn the single end reward into a per-step advantage.

Q: What do the confidence intervals mean?
A: 95% bootstrap intervals: resample the test scenes 2,000 times and take the middle 95% of averages. For BC and PPO each (seed, scene) is resampled, so seed variation is included. Per-seed numbers are in the seed table.

Q: Why is UNDO almost never used?
A: After a jitter the object is usually one cell from where it should be. A normal move fixes that for the same price as UNDO, and sometimes makes progress at the same time. The no-UNDO ablation confirms the planner and BC do not change.

Q: What is the hardest part for the learned policies?
A: Mapping the state to the right object and direction. With noise off they still fail about 5–11% of scenes, while the planner is perfect. In the main setting, of PPO's 643 failed test episodes (over 3 seeds), 327 were STOP with a goal still unmet and 316 ran out of budget; PPO wastes 0.16 actions per episode on invalid moves or empty UNDOs (BC: 0.29, planner: 0.00). Details: `results/failure_analysis.json`.

Q: Could PPO match the planner with more training?
A: Maybe a little: the learning curves had mostly flattened after about 2 million steps. But the planner has a model and searches; a reactive network would likely need more capacity or a better input design to fully close the gap.

10 Limitations and deviations from the proposal

Limitations

- **The puzzles are easy.** Following the proposal's reset recipe, many scrambles move objects no goal cares about, so the shortest fix is often 1 or 2 moves (1 move: 231, 2 moves: 157, 3 moves: 78, 4 moves: 26, 5 moves: 7, 6 moves: 1). Harder resets would separate the policies more.
- **The noise is made up.** 0.7 / 0.2 / 0.1 are modelling assumptions, not measurements of Imagine or any real editor (as the proposal says).
- **Hints.** The observation includes computed hints (legal moves, gaps, pulls). They add no new information but they do make learning easier, and results without them would be much worse.
- **Ablations used fewer PPO steps** (2M instead of 3M), so small PPO differences between settings should not be over-read.
- **The planner is near-perfect** because it has the exact rules. In a real editor the model would be imperfect and the comparison could change.
- **Only three seeds** per learned policy, on one CPU machine. No HPC was used; GRPO (optional in the proposal) was not attempted.
- The frozen parser from language to goals is outside this project: goals are given directly, as the proposal scoped it.

Deviations from the proposal, and why

- **Code layout:** the proposal sketched `rl_env.scene_env.SceneEnv`; the code is `scene_repair.env.SceneRepairEnv` with the same Gymnasium five-value interface. The old public repo (rl-scene-env) still holds the earlier one-step environment and was not changed.
- **Planner search:** A* with an admissible estimate (allowed by "BFS or greedy" in spirit: it finds the same shortest plans as BFS, just faster), with an expansion limit of 20,000 per decision. On the test set it hit that limit 0 times in 7,239 decisions (the largest single search expanded far fewer layouts).
- **Undo buffer rule detail:** every MOVE, including failed and invalid ones, saves the pre-move layout (a literal reading of "a MOVE saves its pre-action layout"). So UNDO after a failed move is a charged no-op.
- **"Template" split:** a template is defined as the exact goal list. Hashing it decides the split, so no goal list is shared between train, validation and test.
- **Padding choice** (the proposal asked to document it): 22 actions always; missing-object actions masked, and charged as invalid if chosen anyway.
- **Number of goals and objects:** the proposal does not fix them; we use 3–5 objects and 2–4 goals, at least one of them relational.
- **Added observation hints** (pull, gap, legal-move flags), for the reason given in section 6.4.

Tiny glossary

Policy: the rule that picks an action from a state. **Episode**: one puzzle from start to STOP or empty budget. **Advantage**: how much better an action turned out than expected. **Value head**: the part of the net that guesses the final reward. **Entropy**: how spread-out the action probabilities are (high = more random). **One-hot**: zeros with a single 1 marking a value. **A***: a search

that explores the most promising options first using a safe estimate of the remaining cost. **Bootstrap CI**: a confidence range found by resampling the data many times. **Ablation**: turning one feature off to see how much it matters. **Seed**: the number that fixes the randomness so a run can be repeated.